



STOPS – A Graph-Based Study Planning and Curriculum Development Tool

Tapio Auvinen[†], Juha Paavola^{*} and Juha Hartikainen^{*}

Aalto University

[†]Department of Computer Science and Engineering

^{*}Department of Civil and Structural Engineering

Espoo, Finland

firstname.lastname@aalto.fi

ABSTRACT

STOPS (Software for Target-Oriented Personal Syllabus) is a tool that allows university students to create personal study plans and the staff to maintain and develop course contents and curriculum structure. In the system, the curriculum is modeled as a graph of learning outcomes. First, the courses are broken down to learning outcomes, i.e. what a student should know after completing a course. Second, prerequisite dependencies are defined between the outcomes. Finally, programmes are defined as goals that students can choose for their studies. Programmes consist of competence outcomes that have prerequisite dependencies from the outcomes of the courses. The resulting graph can be visualized so that students can see how the topics taught in various courses are connected to each other, and what each course contributes to the goals that the student has selected. For curriculum developers, the visualizations can reveal shortcomings such as unnecessary repetition or lacking coverage of topics in the course contents.

Categories and Subject Descriptors

J.1 [Administrative Data Processing]: Education

General Terms

Experimentation, Management, Design

Keywords

curriculum development, curriculum design, curriculum visualization, curriculum mapping, study planning

1. INTRODUCTION

University studies often offer students a lot of freedom in choosing what to study and how to schedule the courses. However, this freedom requires students to take responsibility for properly planning their studies as poor planning may lead to prolonged studies or a selection of courses that do not serve a purpose. It is, therefore, important that students are offered proper tools and support for preparing personal study plans.

In this paper, we present the *STOPS* system (Software for Target-Oriented Personal Syllabus) which serves two purposes: it allows teaching staff to design a curriculum model and students to construct personal study plans. The curriculum model consists of *programmes* that students can choose as goals for their studies, *courses* that the university offers, *outcomes* that make up the courses and programmes, and *prerequisite dependencies* that connect the outcomes. This curriculum model results in a graph of learning outcomes that represents the hierarchical and cumulative nature of the knowledge and skills covered in courses. The system can visualize the curriculum model in order to give both the students and the curriculum developers a better understanding of its structure.

The main purpose of the tool is to make the structure of the curriculum visible to students. Instead of giving them a plain list of mandatory courses, the tool allows students to see where the topics covered in each course are going to be needed in the subsequent courses and how they contribute the goals selected by the student. Curriculum developers, on the other hand, can use the visualizations to identify shortcomings in the course contents or the curriculum structure.

The rationale behind the concept has previously been introduced in [18] and a work-in-progress implementation in [1]. In this paper, we describe the *STOPS* system in detail and report the results of an evaluation study. The system was used in designing the new bachelor's program of the *School of Engineering at Aalto University* and made available for the freshmen students for preparing their personal study plans. We conducted a controlled experiment where half of the students were shown curriculum visualizations and half used course lists similar to traditional study guides, and analyzed from log data how the students used the tool.

This paper is structured as follows: Related work is discussed in Section 2. The system is described in Section 3 and the results of the evaluation study in Section 4. In Section 5, we present our interpretation of the results. Finally, Section 6 concludes the paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

Koli Calling '14, November 20 - 23 2014, Koli, Finland

Copyright 2014 ACM 978-1-4503-3065-7/14/11 ...\$15.00

<http://dx.doi.org/10.1145/2674683.2674689>

2. RELATED WORK

2.1 Curriculum development

McKenney et al. define curriculum development as an activity directed at (re)designing, developing and implementing plans for learning [13]. It typically includes analysis, design, construction, evaluation and implementation phases [13]. Curriculum development is a broad concept that can cover anything from designing the high-level goals of degree programmes to the contents of individual lectures. In this paper, our focus is on computer-assisted tools for designing the outcomes of courses and programmes.

An important part of curriculum development is *curriculum mapping*, which refers to mapping course contents to programme-level goals. According to English, it reveals what is actually being taught, how long it is being taught, and the match between what is being taught and the district's testing program [4]. A typical way to perform curriculum mapping is to construct a matrix with courses as rows and programme-level goals as columns, and tick the appropriate cells of the courses addressing each goal. For example, in the process described by Sumsion and Goodfellow [24], unit coordinators were tasked to indicate how generic skills (e.g. communication skills, problem solving, etc.) are fostered in their respective units. The produced map offers the academic staff an understanding of what generic skills are fostered and what are overlooked in the curriculum. Furthermore, it gives the teachers an opportunity to reflect on their course and consider ways to develop the them.

Some computer-assisted tools have been developed to support curriculum development and curriculum mapping. *CASCADE* [13] is a family of tools that support curriculum designers in various phases of educational curriculum materials development. It is mostly focused on planning, creating and evaluating learning materials and lesson plans. *COMET* [23] supports creating competence maps, i.e. the documents describing the final attainment levels that define the program of study, by offering templates and scaffolding for describing and ordering competences. *CCMap* [17] is an Excel-based curriculum mapping tool which provides visual representations of various aspects of the curriculum. It provides a template where teachers tick which graduate attributes are addressed in their respective course, and then provides charts of the emphasis of the different attributes in the curriculum. It also takes into account the level of thinking using the Krathwohl's Revised Teaching Taxonomy [10], as well as assessment tasks and student activities (reading/writing/practice/etc.) *CUSP* [7] is a system that facilitates curriculum mapping to multiple learning goal sets. The tool allows teachers to map course contents to institutional goals which, in turn, can be mapped to multiple external goal sets that may originate from different accreditation, regulatory, and professional bodies. This makes it possible for degree program coordinators to see whether the courses form a whole that fulfills the goals of the degree programmes. Furthermore, it helps teachers to design learning activities so that the course serves the goals of the programme. Finally, it makes it possible for students to see the purpose and real-world relevance of the contents of each course.

A concept related to curriculum mapping is *core curriculum analysis*. Core curriculum can be defined as the essential or key aspects of the subjects studied in the curriculum, or those aspects that are common to all specialties in the

curriculum [8]. It can be utilized in curriculum design by, for example, first defining the goals of a degree programme, then categorizing the goals into core and specialties, and finally mapping course contents so that the core curriculum is covered in mandatory studies and specialties in the elective courses [11]. Furthermore, it may be utilized at the course level to set the assessment criteria so that the core content is required for a passing grade while mastery of the specialized content yields higher grades [12].

The *STOPS* system was inspired by the university's plan to perform core curriculum analysis for course contents by classifying topics into *must know*, *should know* and *nice to know* categories. We argue that different topics are important for different students depending on what other courses they are going to take in the future according to their study plan. Defining prerequisite dependencies between the learning outcomes of the courses makes it possible to evaluate the importance of course topics from the point of view of subsequent courses rather than as a static categorization. This way, knowing the student's goal makes it possible to construct a *personal core curriculum analysis*, i.e. a set of topics that are important for a particular student for reaching the goals stated.

2.2 Curriculum visualization

Several tools exist that visualize the prerequisite dependencies between the courses in a curriculum. Nuutinen and Sutinen [16] present a tool that visualizes the curriculum as a concept map where each node represents a course. However, the tool is limited to displaying courses and their prerequisite relationships and not the contents of courses. *SUPSI 3D* [22] visualizes curricula as 3D graphics. Departments, degree programs and semesters are rendered as regions in space and courses as boxes of different sizes. Numerical properties of courses, such as credits and duration, determine the dimensions of the course boxes so that the user can easily discern the characteristics of different courses. The system focuses on the visualization of whole degree programmes, without going into details such as learning outcomes of courses or the prerequisite relations between courses. *ViCurriAS* [26] allows staff to construct curriculum maps, i.e. graphs of courses that are connected by prerequisite dependencies. This allows instructors to examine how changes in courses affect the whole curriculum. The system can also be used for student counseling as it allows tracking their progress. The system does not, however, show the learning outcomes of the courses but deals with prerequisite dependencies at the course level. *CurricViz* [6] visualizes prerequisite dependencies between courses as directed graphs. The tool helps students to choose courses or programs, and visualizes their progress. However, it does not display the contents or learning outcomes of the courses. Kabicher and Motschnig-Pitrik [9] present a wiki-based curriculum planning tool that automatically creates visualizations of module dependencies. The application is aimed for participatory curriculum design where instructors can collaboratively design how contents are divided between courses. However, the prerequisites cannot be modeled between individual learning outcomes. Moreno et al. [15] propose using circular diagrams to visualize dependencies between courses. The authors observed improved correctness when students made term by term study plans using the new visualization. Rollande and Grundspenikis [19] have implemented a person-

alized study planning system that models the curriculum as graphs. The curriculum is modeled as courses that can have prerequisite dependencies. Each course, in turn, contains a graph of topics, each topic a concept map, and each node in the concept map is connected to learning objects. The system can be used by staff to design course structures, and by students to design individual study programmes. However, the system does not map course topics to the goals of degree programmes.

It is common to all of the previously discussed visualization tools that they model connections between whole courses instead of individual learning outcomes. With such visualizations, students do not get a picture of how the topics taught in courses are connected to other courses or programme-level goals. Ontological models offer one way to model the curriculum so that the fine-grained connections between course contents are taken into account. Dexter et al. [3] report on a work-in-progress project where an ontological model of a medical curriculum is created. The ontology maps *Intended learning outcomes* (ILOs) to *competencies* which are complex mixtures of knowledge, skills and personal qualities. Their aim is to create a knowledge base system which allows them to tackle the complexity of the interrelationships between the component parts of the curriculum when it has to be continuously updated. However, this approach is not aimed as a study planning tool for students but rather for developing and documenting curriculum contents for administrative purposes.

Siirtola et al. [21] have developed a curriculum visualization tool that visualizes the overlap of courses. In their process, teachers first list the topics covered in their courses. The topics are then automatically matched and the overlap is visualized as node-link diagrams. The tool can also visualize the distance between courses as a matrix representation which allows users to identify clusters of thematically similar courses. The tool is useful for example, in comparing course offerings between two institutions in case of a merger or collaboration. It does not, however, deal with the prerequisite dependencies or the order in which courses should be taken and is thus not sufficient as a study planning tool for students.

One issue in curriculum visualization is that the course descriptions are typically written in natural language which makes it difficult for a machine to automatically recognize similarities or connections between courses. Gatteschi et al. [5] address this issue by using the WordNet semantic thesaurus [14] to find the relations between the words in course descriptions. Similar words are clustered to produce a representation that hides the differences in terminology. This allows users to compare similarities of course contents between institutions. Sant et al. [20] in turn, use Multi-Dimensional Scaling (MDS) to produce a 2D map visualization from word frequency vectors that are extracted from course descriptions.

3. DESCRIPTION OF THE SYSTEM

STOPS is an online tool that can be used with the browser. It has an interface for curriculum developers for constructing the curriculum model, and an interface for students for browsing curricula and constructing personal study plans.

The curriculum model consists of *courses* and *programmes*, their *outcomes*, and *prerequisite dependencies* between the outcomes. Courses consist of learning outcomes, i.e. knowl-

edge or skills that students should possess after completing the course. The learning outcomes are described in natural language. For example, the *Statics and Dynamics* course has a learning outcome *"Student is able to understand basic concepts of force, moment, momentum and angular momentum and express them in scalar and vector form."*

Prerequisite dependencies are defined between the learning outcomes of different courses. For example, the previous example from Statics and Dynamics has a prerequisite *"Student is able to represent a system of linear equations in matrix form"* from the *Matrix algebra* course. A prerequisite can be either *mandatory* or *supporting*. The mandatory prerequisites define the order in which courses must be taken while the supporting prerequisites can be used to visualize how topics are connected, and to recommend elective courses.

Programmes represent larger wholes that students can choose as goals for their studies. In our current implementation, a *programme* can be a major or a minor of a bachelor's or master's programme, but they could represent more informal structures as well. *Programmes* consist of *competence outcomes* in the same way as *courses* consist of *learning outcomes*. The outcomes of programmes can have prerequisite dependencies from the outcomes of courses or other programmes. For example, the *Energy and Environmental technology* major has outcomes *"Geoengineering and geology"* and *"Personal planning and project management skills"*, among others. These outcomes are linked to the learning outcomes of the courses where these topics are taught or practiced. The outcomes can be concrete substance knowledge as well as generic skills such as project management, team work or presentation skills. The connections between the outcomes of the programmes and courses form a *curriculum mapping*, but in addition to direct links between the courses and programme-level goals there are paths of learning outcomes that span over multiple courses.

Early attempts to describe the learning outcomes suffered from different teachers describing them with varying terminology and granularity. In order to produce more uniform learning outcome descriptions and to take into account the intended level of proficiency, teachers are now instructed to utilize Bloom's taxonomy [2] and use verbs such as *student recognizes*, *understands*, *can apply*, *can analyze* or *can develop* something.

3.1 Architecture

Figure 2 shows the essential parts of the system's class diagram. Attributes such as name and metadata fields, as well as auxiliary classes, are omitted.

Curriculum is a basic container for the courses and programmes that belong to some scope such as a department. This makes it possible to maintain multiple versions of the same curriculum in parallel. For example, revised versions of the curricula can be developed yearly without interfering with students currently using the previous version.

CurriculumNodes are entities that consist of *Outcomes* and can be selected by students to their study plans. Its subclasses are *CurriculumCourse* and *CurriculumProgramme*. Technically, there is little distinction between courses and programmes. In this way, courses and programmes can form arbitrary prerequisite chains. For example, a master's level course may have a bachelor's level major programme as a prerequisite.

Prerequisites

MS-A010x Differential and Integral calculus 1		Course
Student can analyze the convergence of sequences and series	Supporting	Mandatory
Recognizes the series expansions and approximations of elementary functions	Supporting	Mandatory
Understands the most important properties and applications of the derivative and integral	Supporting	Mandatory
Student can solve a first degree separable and linear differential equation	Supporting	Mandatory
Understands the structure of solution of the linear second degree differential equation	Supporting	Mandatory

MS-A020x Differential and Integral calculus 2		Course
Student can calculate approximations using the derivative	Supporting	Mandatory
Student can solve systems of equations using Newton's method	Supporting	Mandatory
Understands the basics of optimization	Supporting	Mandatory
Can use Lagrange multipliers	Supporting	Mandatory
Can change the integrating order in a surface integral	Supporting	Mandatory
Can change variables in surface and volumetric integrals	Supporting	Mandatory
Can calculate surface and volumetric integrals numerically.	Supporting	Mandatory

Learning outcomes

KJR-C1001 Statics and dynamics

Course

- ★ Student is able to understand basic concepts of force, moment, momentum and angular momentum and express them in scalar and vector form.
- ★ Student is able to draw a free body diagram and formulate equilibrium equations.
- ★ Student is able to describe the motion of particles and rigid bodies i.e. velocity and acceleration, in different coordinate systems.
- ★ Student is able to apply equations of motion to describe planar motion of rigid bodies.
- ★ Student is able to analyse systematically real structures and machines by modelling them as particles and rigid bodies in equilibrium and in motion with application of principles of mechanics.
- ★ Student is able to solve problems of mechanics by applying mathematical methods of linear algebra and theory of differential equations.
- ★ Student is able to analyse with criticism the reality of the obtained results.

+ Add outcome

Figure 1: Prerequisite editor. User is specifying the prerequisites of the *Statics and dynamics* course.

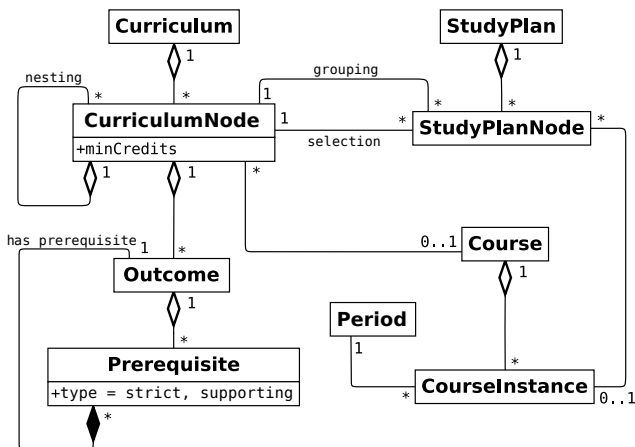


Figure 2: UML class diagram of the essential parts of the system.

Prerequisites are never defined directly between courses or programmes but only between *Outcomes*. An *Outcome* has references to its prerequisites through the *Prerequisite* model which specifies the prerequisite type (mandatory/supporting). The fact that *CurriculumNodes* and their *Outcomes* are scoped to a *Curriculum* results in that courses can have different prerequisite chains in different curricula.

The *Course* class represents a course that is offered at the university but it does not specify its position in different

curricula or when it is arranged. A *CourseInstance*, in turn, defines in which semesters or teaching periods the course is offered. *Periods* are defined as time intervals with starting and ending timestamps which makes it possible to arbitrarily divide the academic year. For example, *Aalto University* recently switched to dividing the academic into five instead of four teaching periods, which poses no problem when each period is defined as a separate object.

StudyPlan represents the study plan of a student, and it is simply a collection of *StudyPlanNodes*, i.e. programmes and courses that the student has selected. A *StudyPlanNode* that is tied to a *CurriculumCourse* can be associated to a *CourseInstance* in order to schedule the course. One course may be mandatory in multiple programmes, and in this case a student is typically required to select elective courses to reach the required number of credits (i.e. the same course cannot be counted twice towards the total credits). To this end, a *StudyPlanNode* representing a course can also be *grouped* to a programme to which the credits will be attributed.

University curricula often allow students to select courses from limited sets of alternative courses as long as the required number of credit points are reached. For example, a student may have the freedom to choose between *Statistics* or *Discrete mathematics*. This scenario can be modeled with *nested programmes*. A nested programme does not appear in the graphs but it creates a group of alternative courses in the list of required courses. The structure is illustrated in Figure 5. The list of mandatory courses required by the parent programme is collected by recursively following the mandatory prerequisites of its outcomes. Lists of alterna-

KJR-C2001 Foundations of Solid Mechanics (5 cr) [+ Add to plan](#)

Yksilöotteina rakenteina mallinnettavien kappaleiden kuten sauvan, palkin ja paineastian rasiustilan määrittäminen vedossa, puristuksessa, taiputuksessa, leikkauksessa ja väännössä. Lagrangen esitystapaa ja lineaaritermoelastista ainemallia käyttäen. Yksilöotteina rakenteina mallinnettavien kappaleiden stabiilustarkastelun perusteet.

Teaching period: IV-V

Learning outcomes:

- osaa määrittää palkin veto-, puristus-, leikkaus-, taiputus- ja vääntörasitukset ja niitä vastaavat muodonmuutokset voiman, momentin, muodonmuutoksen ja jännityksen peruskäsitteitä soveltaen
- ymmärtää virtuaalisen työn periaatteen peruskäsitteet ja osaa soveltaa niitä palkin rasiustusten, muodonmuutosten ja siirymien määrittämisessä
- ymmärtää palkin stabiilustarkastelun perusteet

Links: [Noppa Oodi](#)

Course Graph

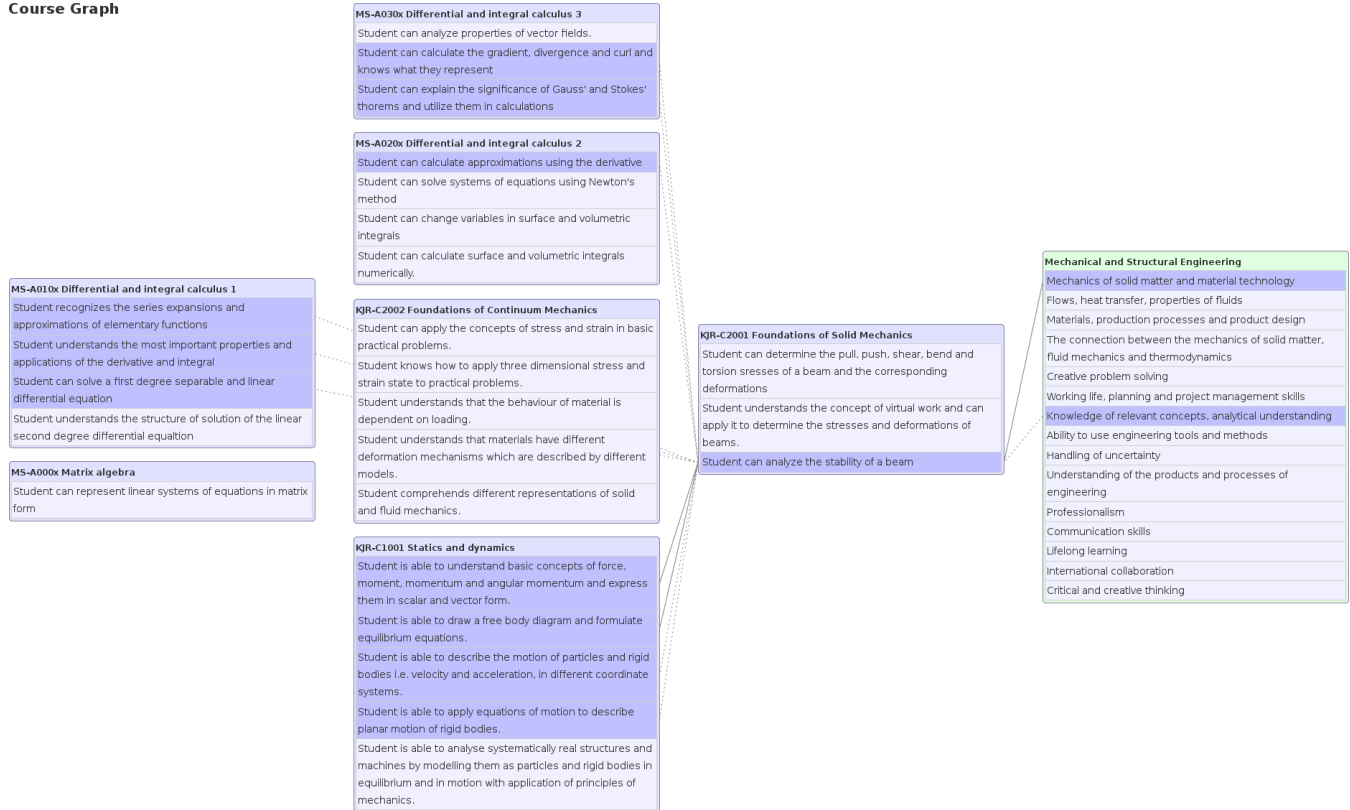


Figure 3: Interactive curriculum graph visualization for the *Foundations of solid mechanics* course. User has clicked the *"Student can analyze the stability of a beam"* learning outcome and the outcomes connected to it have been highlighted. Prerequisite courses are on the left and the *Mechanical and Structural Engineering* programme on the right. Mandatory prerequisites are marked with solid lines and supporting ones with dashed lines.

tive courses, in turn, are collected by following the supporting prerequisite links of the nested programmes. Finally, elective courses can be suggested from the supporting prerequisites of the parent programme.

3.2 Curriculum design

The curriculum model is built with an interactive editor in the browser (Figure 1). On the right, the user can add, remove, edit and reorder the outcomes of the course or programme being edited. To set the prerequisites of an outcome, the user first selects it on the right, then performs a search on the left using keywords, and finally marks the desired outcomes as mandatory or supporting prerequisites.

A typical task for a teacher is curriculum mapping, i.e. specifying which outcomes of the degree programme does their course serve. To support this task, the system also contains a reverse prerequisite editor. In this mode, the teacher sees their own course on the left and the programme on right, and can set prerequisite links between the outcomes of their course and the programme.

Staff can view curriculum visualizations similar to that

shown in Figure 3. The visualization shows all the courses that are prerequisites of the selected course/programme as well as the courses and programmes that have the selected course/programme as a prerequisite. The visualizations can reveal possible shortcomings in the curriculum:

- A competence outcome of a programme is not sufficiently covered in the courses.
- A course has outcomes that serve no purpose as a prerequisite to another course or a programme.
- There are prerequisite loops.

In addition, the curriculum model gives teachers a good understanding of what is taught in other courses so that they do not erroneously assume that something is taught when it's not, and on the other hand, so that they do not unnecessarily repeat topics that have been sufficiently covered already. Furthermore, curriculum developers get a view of how the curriculum is affected if topics are removed or moved from one course to another.

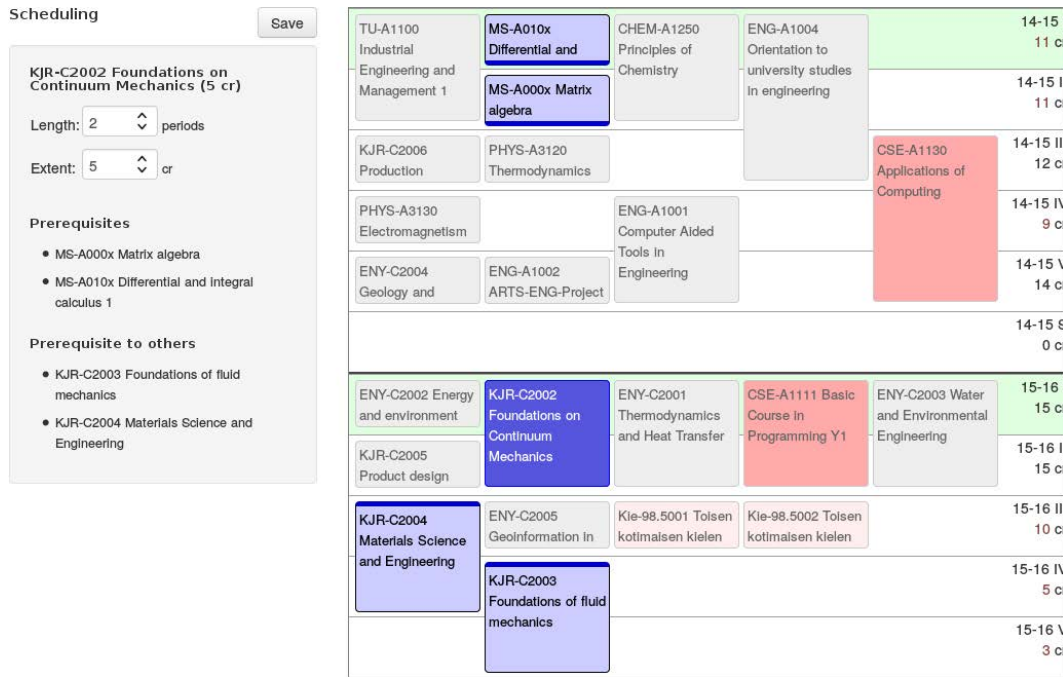


Figure 4: Student's course scheduling view.

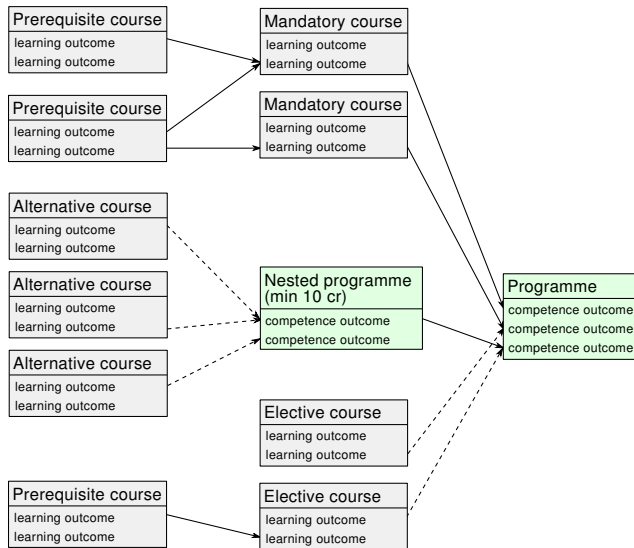


Figure 5: Example of a curriculum structure.

3.3 Study planning

The students' interface consists of three main areas:

- *Curriculum view* for browsing and selecting programmes and courses,
- *Course search and course list view* for viewing the list of selected courses and searching courses by keywords,
- *Scheduling view*

In the curriculum view, students can browse available programmes. After navigating to a programme, the student can see its competence outcomes and the list of mandatory and elective courses that are required to reach the intended

outcomes. By clicking an outcome, a student will get a list of courses that support this particular outcome. In this way, the supporting prerequisite links are used for suggesting elective courses that students might want to take if they are interested in a particular topic within the programme. Below the learning outcomes and the course list, the student can view an interactive graph visualization (similar to that in Figure 3) that contains all the courses related to the programme. Clicking an outcome of the programme highlights its prerequisites. This visualizes to the student which learning outcomes of the courses contribute to the goal and form the *core curriculum* for this goal.

From the course list, the student can navigate to individual courses to view their details (Figure 3). This view shows basic information about the course such as a description, credit points, teaching period. There is also an interactive graph visualization that shows the prerequisites of the course, courses which have the selected course as prerequisite, and the programme inside which the course is being viewed. Clicking any learning outcome highlights the learning outcomes of other courses or programmes that are connected. This allows students to see how the topics taught in different courses are connected to each other and where are the studied topics going to be needed in the future.

In the scheduling view (Figure 4), students can plan the order and timing of the selected courses. In this view, the rows represent teaching periods (in our university, the academic year consists of five teaching periods). The student can drag'n'drop each course to the desired period. Clicking a course highlights the periods in which the course is arranged as well as the courses that are prerequisites or require the selected course as a prerequisite. Dragging a course to a position where the prerequisites are not satisfied causes the affected courses to be highlighted (the red courses in the figure). Initially, each course is automatically scheduled to the earliest period in which its prerequisites are satisfied.

Table 1: How many students made a plan.

	control	treatment
plan	45	36
skip	11	18
total	56	54

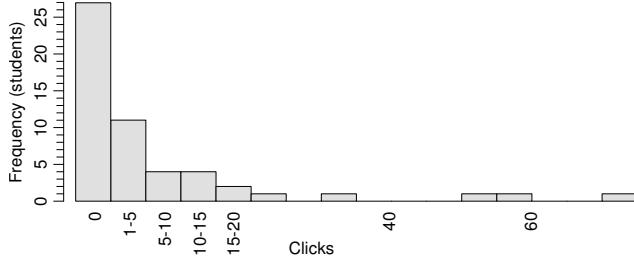


Figure 6: Histogram of the number of clicks in the interactive graphs.

4. EVALUATION

4.1 Methods

The system was used in 2013 in designing the renewed bachelor's programme of the *School of Engineering at Aalto University*. In Fall 2013, it was made available for the new freshmen for designing their personal study plans. In the first year, the freshmen have a mandatory *Orientation to university studies* course where they must prepare and submit a personal study plan. The *STOPS* tool was introduced for the students as an option for study planning, but they were also allowed to prepare the plans using traditional tools such as the printed or online study guides, which do not contain visualizations. *STOPS* was demonstrated in a lecture and advertised in the home page and the email newfeed of the course.

Students who logged into the system were randomly divided into treatment and control groups. The control group was shown only elements that are available in the traditional printed or online study guides, i.e. course lists and written course descriptions. The treatment group, on the other hand, was shown the interactive graph visualizations in addition. The course search and the scheduling views were similar for both groups. The system recorded students' activities such as page views and clicks in the interactive visualizations.

After the study plan contained 145 credit points¹, the student was directed to a feedback survey with open-ended questions about how they used the graphs and how the system could be improved. The threshold was selected so that the student must have a selected major and a few elective courses before taking the survey. However, because only 12 students gave open ended feedback and the answers were very short, the survey answers are not analyzed in this paper. After submitting the survey, the student was removed from the experimental setting and given full functionality of the system. The experiment ended in the end of the semester, and during the next semester, all students had the full functionality.

¹A complete bachelor's level study plan contains 180 cr. Credit points (cr) are measured in ECTS points (European Credit Transfer System).

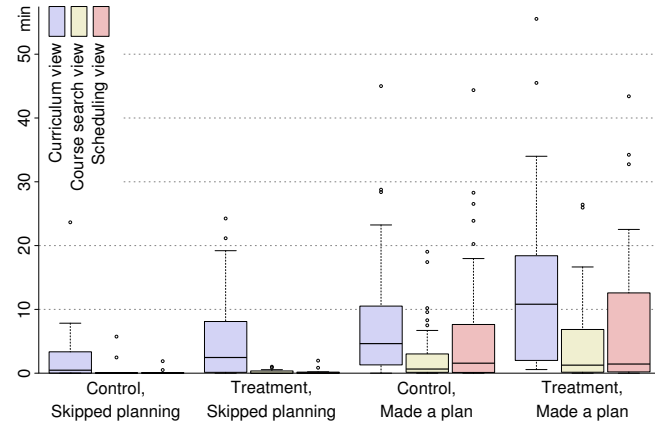


Figure 7: Time used in different views.

4.2 Results

4.2.1 Curriculum development phase

The tool was used at the *School of Engineering at Aalto University* when designing the renewed bachelor's programme. The department has three majors: *Energy and Environmental technology*, *Mechanical and Structural Engineering*, and *Built Environment*. The teaching staff of the programme designed completely new course contents and curriculum structure during the 2012-2013 academic year and the new curriculum was used by the new freshmen in Fall 2013.

In the planning phase, the academic staff planned the new curriculum in workshops to agree on the goals of the bachelor's programme and on what courses are needed. Next, the teachers responsible for each course were tasked to enter the learning outcomes of their course to the *STOPS* system. After the learning outcomes were entered, teachers specified the prerequisites of their course and created the connections between their course and the outcomes of the programme. These definitions were iteratively refined in workshops and individually. Curriculum visualizations were used for making sure that the goals of the programme were sufficiently covered in courses, and that the courses formed meaningful paths of learning outcomes for students. The final curriculum model had 59 courses with on average 7.1 learning outcomes per course (median 7, standard deviation 3.5), and 22.8 prerequisite connections per course (median 5, sd 44.2). The average total time used for editing a course was 82.3 minutes (median 63.3, sd 73.3), and the average number of sessions was 23.1 (median 21, sd 13.0)².

4.2.2 Student use of the tool

The orientation course had a total of 279 participants. Two students who had prior studies in the university and were now changing department are excluded from the results. The analysis covers the $N = 110$ students (40 % of the population) who logged into the system at least once during the experiment.

64 % of the students who logged into the system made a study plan. A student is considered to have made a plan if it contains at least 100 credit points worth of courses. This

²Spent time is estimated from the time differences between consecutive events. More than two hours of inactivity is filtered and considered to start a new session.

Table 2: Time used in different views by those who made a plan or skipped planning. Durations are in seconds. Statistical significance was tested with a two-tailed Wilcoxon rank-sum test.

view	group	mean control	mean treatm	W	p-value
curriculum	All	429.911	655.500	1822.5	0.064
	Plan	481.667	804.889	1050.5	0.023
	Skip	218.182	356.722	124	0.267
courses	All	133.947	188.463	1541	0.864
	Plan	155.511	276.778	946	0.197
	Skip	45.7273	11.8333	105	0.790
schedule	All	296.911	309.352	1457.5	0.743
	Plan	366.111	458.167	863	0.617
	Skip	13.8182	11.7222	99	1

threshold is selected because there is only one plan in the range of 21...108 credits but an even distribution of plans with 109...180 credits. Table 1 shows how many students made a study plan in treatment and control groups. The number is slightly higher in the control group but the difference is not statistically significant (Fisher’s Exact Test, $p\text{-value} = 0.131$).

Figure 6 shows a histogram of the students’ total number of clicks in the interactive graphs. Only students in the treatment group are included because there were no interactive graphs for the control group. 27 students (50 % of treatment) did not click the graphs at all. The mean number of clicks for those who clicked at least once was 7.3, and the maximum was 74.

Table 2 and Figure 7 illustrate how much time students spent in different views. Time usage is calculated separately for those who made a plan (> 100 cr) and for those who skipped planning. Students who were in the treatment group and made a plan spent more time in the curriculum views than those who were in control and made a plan. Curriculum views are the ones that contain the interactive graph visualizations and differ between treatment and control. The difference is statistically significant (Wilcoxon rank-sum test, two-tailed, $p=0.023$). They spent more time in the other views as well but the differences are not statistically significant. Furthermore, students in treatment who skipped planning spent more time in the graph view than their counterparts in control, although the difference is not statistically significant.

Table 3 shows the number of sessions, total time spent in the system, and the average time spent viewing the pages of individual courses. Students in treatment who made a plan spent statistically significantly more time in course pages and in total.

4.2.3 After the experiment

The tool was available for students during the next semester as well, and graphs were visible for all students. During the next semester, 21 students (19 % of the users) continued to use the tool, even though it was not advertised in any way. In addition, there were 6 new users who had not logged in during the experiment. Slightly more students from the control group continued to use the tool (Table 4) but the difference is not statistically significant (Fisher’s exact test, $p=0.334$).

Table 3: Number of sessions, total time used and average course viewing time (in seconds) of those who made a plan or skipped planning. Statistical significance was tested with a two-tailed Wilcoxon rank-sum test.

variable	group	mean control	mean treatm	W	p-value
sessions	All	1.85714	2.09359	1590.5	0.612
	Plan	2.00000	2.47222	936	0.205
	Skip	1.27273	1.33333	95.5	0.856
total time	All	1090.23	1437.26	1733	0.187
	Plan	1279.33	1917.89	1018	0.049
	Skip	316.636	476.000	135	0.111
course time	All	25.7703	41.7910	1635	0.025
	Plan	24.1717	38.7408	1034	0.020
	Skip	35.8185	49.1113	54	0.945

Table 4: How many students used the tool during the next semester.

	control	treatment
used	13	8
not	43	46

Those who had been in the treatment group used more time in the graph views, although the difference is not statistically significant (Wilcoxon rank-sum test, two-tailed, mean treatment=110.875 s, mean control = 92.0 s, $W=62.5$, $p=0.456$). Statistically significant differences are, however, observed in the number of clicks in the graphs (Wilcoxon rank-sum test, two-tailed, mean treatment=4.37500, mean control=0.07692, $W=76$, $p=0.023$).

5. DISCUSSION

Constructing the curriculum model is a laborious task, and some teachers were not happy about the required work. However, we argue that documenting the learning outcomes of the courses is a task that is necessary in any curriculum design procedure. Marking the prerequisite dependencies between the learning outcomes requires some additional effort, but we believe that the benefits justify the cost. The average time teachers used in the system was less than two hours per course, although they may also have spent time planning outside of the system, which is not revealed by the logs.

When connections between learning outcomes are modeled in detail, the curriculum structure becomes transparent to students. Study plans are now constructed top-down so that student selects a competence goal, and the list of required courses is generated based on the prerequisite chains. This way, the study plan appears to students as a graph of interconnected learning outcomes where every course serves a purpose, rather than an arbitrary list of courses.

The original purpose of the tool is to increase student’s motivation and engagement with their studies by helping them to see what each course contributes to the goals that they have selected for studies. However, measuring such effects is difficult, especially when the feedback survey had very low response rate. In this paper, we are limited to observing some aspects of student’s recorded behavior in the system.

Out of the students who made a study plan, those in treatment group used 67 % more time ($p=0.023$) in the curriculum views, which are the views that contain the graph visualizations. Time usage does not tell unambiguously whether the graphs are useful for students or not. One could argue that increased time usage is a sign of inefficiency. However, students in the treatment group would have been able to prepare their study plans in the same way as the students in the control group by simply ignoring the graphs. Thus, the increased time usage tells that students found the graphs interesting enough to spend time to study them.

The number of clicks in the interactive graphs was low. Half of the students did not click the graphs at all, and the average number of clicks was 7 for those who did. The high number of students who did not click at all could indicate a usability problem where students did not realize that the items can be clicked. However, there was an instruction text and the usage was demonstrated in a lecture before the system was released to students.

The low number of clicks among those who did click the graphs could indicate that the students did not find exploring the graphs very useful. We believe that the graphs are currently too complex for students. The system was first used by curriculum designers to plan course contents and then by students to create study plans. However, while the designers need a complete and detailed model of the curriculum, students would probably benefit from seeing only a few main learning outcomes for each course. Ideally, a student would see a clear path of learning outcomes from the selected course through subsequent courses to the programme but currently, because of the complexity of the model, connections can only be shown comprehensibly between the selected course and its immediate neighbors. Further research is needed in abstracting the graphs so that only the essential information is shown. One approach could be to implement social navigation by showing the students only the items that other students have frequently clicked.

Differences were seen in students' behavior during the next semester after the experiment. Students from the control group did not click the graphs almost at all (average 0.08 times). During the next semester, the system was mostly used for updating the plan or viewing the schedule instead of selecting courses, and thus the students in the control group probably had no reason to familiarize themselves with the graphs which mostly serve the purpose of choosing courses. On the other hand, students in treatment were already familiar with the graphs and may have been interested to see how the courses offered in spring are connected to their goals.

There may be several reasons why relatively low number of students prepared a study plan with the tool. First, only the bachelor's programmes of the School of Engineering were visible in the system but many students take a minor from another school. Second, master's programmes were not visible in the system because the renewed programmes had not been designed yet. Most students in our university aim for a master's degree, and thus not being able to view the master's programmes may have made the system less appealing. Third, we were not able to release the system to students in the beginning of the semester but some weeks after it was demonstrated to students. Finally, the system is at a prototype stage and requires more polishing.

Study planning seems to be an under-researched topic in educational technology. However, goal-setting is an im-

portant step in self-regulated learning [25] which university studies inherently require. There is a lot of research on computer-assisted systems for curriculum developers that can help to create more or less sophisticated curriculum models. However, there is a gap in bringing these models to the students' disposal. *STOPS* is an attempt to combine in a single tool the curriculum design and study planning aspects. The system is available as open-source software at <https://github.com/Aalto-LeTech/stops>.

5.1 Validity threats

The time spent on each page is estimated from the time differences of consecutive page load events. Other external activities, such as browsing other websites, may lead us to overestimate the time used in the tool. Furthermore, users may be browsing simultaneously in multiple browser tabs which makes it difficult to estimate the actual time spent viewing a page from the time it was loaded.

The graph click events are buffered and sent to the server in bursts in order to reduce server load. This buffering may cause up to 10 seconds worth of clicks just before leaving the page to be lost. Thus, the number of graph clicks may be underestimated. However, the first click event is always sent immediately, thus this does not account for the high number of students without any clicks.

6. CONCLUSIONS

In this paper, we have described a curriculum modeling method and a tool where the curriculum is represented as a graph of learning outcomes. Visualizations of the graph can be used by curriculum developers to identify shortcomings in the course contents, and by students to gain an understanding of how the courses are connected to the goals that the student has selected.

In traditional curriculum mapping methods, there are connections between courses and the programme-level goals. In our method, in contrast, it is possible to map learning outcomes of courses not only to the programme-level goals but also to other courses. Thus, courses can be connected to the programme-level goals through paths of learning outcomes that span over multiple courses.

The system was originally created as a tool to visualize for students how the topics taught in courses are connected to each other and the goals of the student. Our hypothesis is that realizing where the currently studied topics will be used in subsequent courses increases studying motivation. Based on this evaluation study, we are unable to conclude whether students find the curriculum model and graph visualizations beneficial. On one hand, students spent significantly more time viewing courses when visualizations are present, but on the other hand, they did not utilize the interactive visualizations much. More research is required to find out how students experience the tool and how to simplify the visualizations. However, the tool has value as curriculum development tool as well, even without the student's interface. The constructed curriculum model acts as documentation of the course contents and curriculum structure, and the graph visualizations and scheduling provide tools for improving the curriculum.

The best use of this tool will be reached if a wider University network will take it into use. This would give students the possibility to see the available courses and pick up useful courses from other units or Universities to be included into

their study plans. The tool could also be utilized in defining the different learning goals of different level institutions – vocational, polytechnics, universities – to make it easier for students to do most useful choices for their long-term career goals.

7. REFERENCES

- [1] T. Auvinen. Curriculum development using graphs of learning outcomes. In *Proceedings of the First EUCEET Association Conference*, pages 27–36, Patras, Greece, November 2011.
- [2] B. S. Bloom and D. R. Krathwohl. Taxonomy of educational objectives: The classification of educational goals. Handbook I: Cognitive domain. 1956.
- [3] H. Dexter and I. Davies. An ontology-based curriculum knowledgebase for managing complexity and change. In *Advanced Learning Technologies, 2009. ICAALT 2009. Ninth IEEE International Conference on*, pages 136–140, July 2009.
- [4] F. W. English. Curriculum mapping. *Educational Leadership*, 37(7):558–59, 1980.
- [5] V. Gatteschi, F. Lamberti, and C. Demartini. A graphical approach for comparing qualifications. In *Global Engineering Education Conference (EDUCON), 2014 IEEE*, pages 361–366, April 2014.
- [6] P. Gestwicki. Work in progress – curriculum visualization. In *Frontiers in Education Conference, 2008. FIE 2008. 38th Annual*, pages T3E–13. IEEE, 2008.
- [7] R. Gluga, J. Kay, and T. Lever. Foundations for modeling university curricula in terms of multiple learning goal sets. *IEEE Transactions on Learning Technologies*, 6(1):25–37, 2013.
- [8] R. M. Harden and M. H. Davis. Amee medical education guide no. 5. the core curriculum with options or special study modules. *Medical Teacher*, 17(2):125–148, 1995.
- [9] S. Kabicher and R. Motschnig-Pitrik. Coordinating curriculum implementation using wiki-supported graph visualization. In *Advanced Learning Technologies, 2009. ICAALT 2009. Ninth IEEE International Conference on*, pages 742–743. IEEE, 2009.
- [10] D. R. Krathwohl. A revision of Bloom’s taxonomy: An overview. *Theory into practice*, 41(4):212–218, 2002.
- [11] L. M. Levander and M. Mikkola. Core curriculum analysis: A tool for educational design. *Journal of Agricultural Education and Extension*, 15(3):275–286, 2009.
- [12] S. Lindblom-Ylänne and K. Hämäläinen. The Bologna Declaration as a tool to enhance learning and instruction at the University of Helsinki. *International Journal for Academic Development*, 9(2):153–165, 2004.
- [13] S. McKenney, N. Nieveen, and J. Van den Akker. Computer support for curriculum developers: Cascade. *Educational technology research and development*, 50(4):25–35, 2002.
- [14] G. A. Miller. WordNet: A Lexical Database for English. *Commun. ACM*, 38(11):39–41, Nov. 1995.
- [15] C. A. Moreno, W. F. Bischof, and H. J. Hoover. Interactive visualization of dependencies. *Computers & Education*, 58(4):1296 – 1307, 2012.
- [16] J. Nuutinen and E. Sutinen. Visualization of the learning process using concept mapping. In *Advanced Learning Technologies, 2003. Proceedings. The 3rd IEEE International Conference on*, pages 348–349, July 2003.
- [17] B. Oliver, S. Ferns, B. Whelan, and L. Lilly. Mapping the curriculum for quality enhancement: Refining a tool and processes for the purpose of curriculum renewal. In *Australian Universities Quality Forum*, pages 2–3, 2010.
- [18] J. Paavola, T. Auvinen, and J. Hartikainen. Software for Target-Oriented Personal Syllabus – STOPS. Presented at the 2012 AECEF Symposium, London, United Kingdom, September 2012.
- [19] R. Rollande and J. Grundspenki. Graph based framework and its implemented prototype for personalized study planning. In *e-Learning and e-Technologies in Education (ICEEE), 2013 Second International Conference on*, pages 137–142. IEEE, 2013.
- [20] J. Sant. Visualization of curricula using multi-dimensional scaling. In L. Cantoni and C. McLoughlin, editors, *Proceedings of World Conference on Educational Multimedia, Hypermedia and Telecommunications 2004*, pages 1911–1917, Lugano, Switzerland, 2004. AACE.
- [21] H. Siirtola, K.-J. Raiha, and V. Surakka. Interactive curriculum visualization. In *Information Visualisation (IV), 2013 17th International Conference*, pages 108–117, July 2013.
- [22] L. Sommaruga and N. Catenazzi. Curriculum visualization in 3D. In *Proceedings of the twelfth international conference on 3D web technology*, pages 177–180. ACM, 2007.
- [23] A. Stoof, R. L. Martens, and J. J. van Merriënboer. Web-based support for constructing competence maps: design and formative evaluation. *Educational Technology Research and Development*, 55(4):347–368, 2007.
- [24] J. Sumsion and J. Goodfellow. Identifying generic skills through curriculum mapping: a critical evaluation. *Higher Education Research & Development*, 23(3):329–346, 2004.
- [25] B. J. Zimmerman. Investigating self-regulation and motivation: Historical background, methodological developments, and future prospects. *American Educational Research Journal*, 45(1):166–183, 2008.
- [26] R. Zucker. Vicurrias: A curriculum visualization tool for faculty, advisors, and students. *Journal of Computing Sciences in Colleges*, 25(2):138–145, 2009.